

Static Verification of AWS Step Functions with Sound Data-Flow Analysis and Workflow Semantics

Anonymous Author(s)

Anonymous Institution
anonymous@example.org

Abstract. AWS Step Functions workflows are executable JSON artifacts, yet existing validators mostly check syntax and structural well-formedness. Data-flow mistakes, unsafe retries, missing compensations, parallel write races, and unbounded callbacks can therefore deploy and fail only at run time. We present STEP CHECK, a static verifier for Step Functions. Its core is a flow-sensitive analysis of the ASL I/O pipeline that reports a missing-field access only when every execution reaching the read would fail, giving no false positives under the modeled JSONPath semantics. Around this native core, seven further checks cover structural correctness, typed contracts, typestate ordering, retry safety, Saga compensation, conservative concurrency, and temporal bounds. Native ASL facts run immediately; omitted task semantics can be declared or inferred, with severity graded by provenance. Against *three* formal workflow-soundness verifiers (Woflan, BPMN Analyzer 2.0, and BProVe) run as baselines over 193 real AWS workflows, STEP CHECK is complementary—all overlap on control-flow soundness while STEP CHECK alone covers the data-flow, atomicity, concurrency, and temporal classes that have no workflow-net expression; the verifiers over-report 8–29% of valid serverless workflows. On an industrial saga set STEP CHECK runs as a sub-12 ms build-time gate that fails the build on injected defects the platform validator deploys silently. It also rediscovers independently fixed real bugs and detects injected missing-field reads confirmed by a code-disjoint bounded-loop oracle, with every controlled recall reported with an exact confidence interval rather than a bare percentage.

Keywords: Serverless computing · AWS Step Functions · Workflow verification · Data-flow analysis · Saga transactions

1 Introduction

Serverless applications increasingly encode business processes as managed workflows. In AWS Step Functions, the executable artifact is an Amazon States Language (ASL) state machine: JSON for task calls, branches, retries, parallelism, and error handling [4]. These workflows charge cards, reserve inventory, move documents, and coordinate human approvals, so latent workflow bugs become production failures rather than local exceptions.

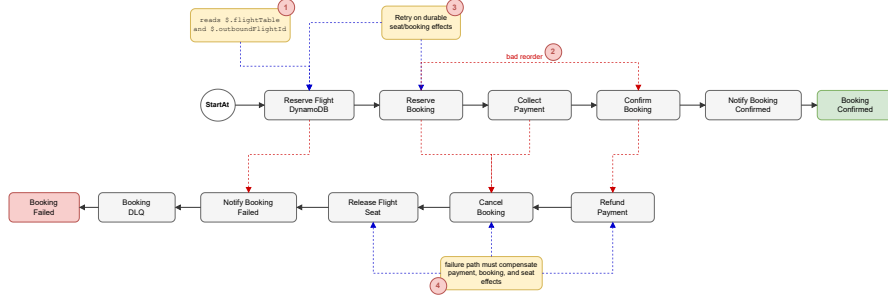


Fig. 1. AWS Serverless Airline Booking **ProcessBooking**, extracted from the public **aws-samples** SAM template [2] and used as a serverless benchmark by Eismann et al. [15]; marker (2) is an invalid mutation, not an upstream transition.

Figure 1 uses a production proxy rather than a synthetic order graph. The application is a widely reused public benchmark: Eismann et al. use AWS Serverless Airline Booking as the canonical case application in their study of serverless performance-test stability [15], a line of work with roughly 30 citations in the serverless literature. In the upstream template, an AppSync resolver builds the execution input from frontend booking fields, but the state machine itself starts at **Reserve Flight**: it reserves a flight seat through DynamoDB, reserves the booking, collects payment, confirms the booking, and notifies the customer. Its failure paths refund payment, cancel the booking, release the seat, and send unresolved failures to a booking DLQ.

The markers show why syntax validation is not enough. (1) frontend/schema drift can make a JSONPath read fail although the graph validates; (2) a one-edge ordering mutation can confirm before payment; (3) retries around durable seat and booking effects are safe only under an idempotency premise that ASL does not encode; and (4) Saga correctness depends on matching persistent effects with compensation paths (Section 2.2). The two remaining classes in **STEPCHECK**—parallel write interference and unbounded callbacks—are evaluated on other public AWS and external workflows, not drawn into **ProcessBooking**, whose upstream ASL has neither a **Parallel** state nor a callback task.

Existing validators check that the JSON is well-formed, transitions exist, and paths are syntactic [5,9,3]. They do not know whether a task produces the field its successor reads, whether a retry is safe for a non-idempotent effect, whether two branches race on one resource, or whether a durable effect is compensated after failure. The missing facts are semantic: task contracts, protocol states, idempotency, write footprints, time bounds, and compensators. Durable-execution systems enforce recovery at run time [10], not before deployment.

We present **STEPCHECK**, a static verifier for AWS Step Functions. Its thesis is simple: verify the *deployed artifact* under *layered semantics*. The unifying contribution is a single workflow IR: its native facts support sound or deliberately under-approximating checks, and the business facts ASL omits are recovered as

provenance-labelled premises. Native facts in ASL support immediate structural, data-flow, temporal, and conservative concurrency checks. Facts ASL omits can arrive from a sidecar, typed DSL, generated metadata, or lightweight inference; STEPCheck grades findings by that provenance, so inferred facts produce warnings while declared facts produce errors relative to the declared model. Adoption therefore starts on raw ASL and strengthens as teams add semantics.

The core technical result is a flow-sensitive JSON-shape analysis for ASL’s I/O pipeline: it reports a missing-field access only when every execution reaching the read lacks the field (Theorem 1). Around it, seven further passes cover structural correctness, typed contracts, typestate ordering, retry safety, Saga compensation, conservative concurrency, and temporal bounds.

Contributions.

- A deployed-artifact verification architecture for ASL with native and declared semantic layers.
- A no-false-positive data-flow analysis for missing JSON fields (under the modeled JSONPath semantics), plus workflow checks for retry, ordering, concurrency, time, and compensation.
- A Rust implementation with raw ASL, typed-DSL, and CNCF Serverless Workflow front ends.
- Evidence on 193 AWS and 66 CNCF workflows, plus 95 from independent repositories: six real bugs rediscovered, 88/126 missing-field mutants detected with all 92 emitted reports oracle-confirmed, and 100% recall on 616 targeted mutants produced by STEPCheck’s mutator.

2 Background and Problem Setting

ASL exposes enough structure for native checks, but hides the business facts needed for deeper workflow verification. This section fixes the small slice of ASL semantics used by the analyses, then states the omitted facts and design requirements.

2.1 The Amazon States Language as a verification target

A Step Functions workflow is a state machine with a **StartAt** state and named states of eight kinds: **Task**, **Choice**, **Parallel**, **Map**, **Wait**, **Pass**, **Succeed**, and **Fail**. A single JSON document is threaded through the graph. Each state may reshape it through the fixed ASL I/O pipeline: **InputPath** selects input; **Parameters** or **ItemSelector** constructs a task payload; **ResultSelector** reshapes the raw result; **ResultPath** merges that result back into the raw input; and **OutputPath** selects the successor document. A key ending in **.\$** reads a JSONPath from the current document (e.g. a **"Payload.\$": "\$.detail.customer"** constructor reads **\$.detail.customer**), and the runtime fails the reference when that path is absent. Current ASL also has workflow variables and JSONata: **Assign**

stores `$var` values for later states, and JSONata fields read `$states.input`, `$states.result`, and workflow variables. STEP CHECK tracks variables and modeled JSONata object/direct-reference fragments; arbitrary expressions collapse to `T`. Fault handling is also explicit: `Retry` re-executes a state on matching errors, `Catch` routes an error to a handler, and `TimeoutSeconds/HeartbeatSeconds` bound long-running tasks and callbacks.

2.2 What Asl omits, and why the defects are latent

Figure 1 uses the AWS Serverless Airline Booking `ProcessBooking` saga. The intended protocol is reserve flight/booking $\rightarrow$ paid $\rightarrow$ confirmed $\rightarrow$ notified, with failure paths through refund, cancel-booking, release-seat, and a booking DLQ. ASL records the graph, JSON reshaping, retries, catches, and native DynamoDB calls. The defect classes in this paper require semantic facts that ASL omits: a task contract $T : A \rightarrow B$ over input/output fields; a typestate edge $s_{\text{in}}^T \rightarrow s_{\text{out}}^T$ in a business protocol δ ; idempotency of external effects; persistence plus a compensator for durable effects; and, for fan-out workflows outside `ProcessBooking`, write footprints and callback bounds.

Because validators inspect only syntax and structural well-formedness, these faults or invalid mutations deploy: each is valid JSON over a valid graph, and the platform retries and routes errors but infers no contract, business protocol, idempotency, write footprint, timeout obligation, or Saga intent.

2.3 Design requirements for an in-place verifier

An in-place verifier must therefore satisfy four requirements. **(R1) In place:** check the deployed ASL artifact at build time, without a new runtime. **(R2) Sound where native:** when a property follows from ASL alone, report only definite faults. **(R3) Semantic premises:** accept omitted facts from sidecars, DSLs, generated metadata, or inference, and grade severity by provenance. **(R4) One framework:** cover data-flow, ordering, retry, concurrency, temporal, and compensation defects in one tool. In summary, control flow and JSONPath movement are native, sound passes. Contracts, typestate, idempotency, persistence, and compensators are declared or inferred. Resources and time bounds are partially native, conservative passes (full matrix in the technical report).

3 Verification Framework

STEP CHECK separates the artifact being checked from the semantic facts available about it: a shared intermediate representation records the control flow and data movement that ASL exposes, and an annotation slot records the contracts, business states, idempotency, persistence, and compensators that ASL omits. The format-agnostic IR lets analyses run unchanged across input languages; layered annotation keeps raw-ASL verification zero-migration; optional name/resource inference gives existing repositories a non-blocking start; and provenance-based

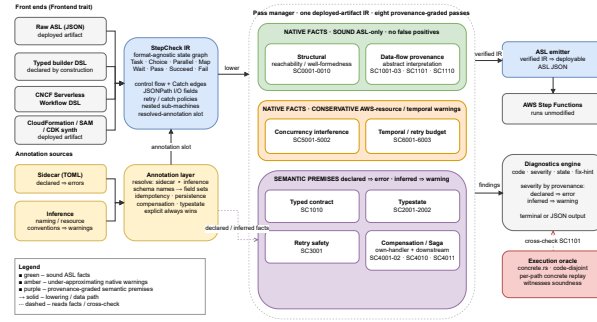


Fig. 2. Verification pipeline: all front ends lower to one deployed-artifact IR shared by the eight passes, rather than running independent lints.

severity keeps CI signals interpretable—native and declared violations error, inferred facts warn.

3.1 Architecture

Every front end lowers to the same workflow graph (Figure 2); every analysis is a pass over it; and the optional emitter lowers verified DSL-authored workflows back to deployable ASL. Raw ASL is the primary input, parsed tolerantly through generic JSON, so intrinsics, context references, SAM placeholders, dialect variants, and JSONata states never cause hard parse failures. The typed DSL emits graph and annotations by construction, and the CNCf front end exercises the same extension point (Section 5.9).

The pass families are summarized in Table 1. Native structural and data-flow checks need only the ASL artifact. *Conservative* native checks report only provable conflicts and may miss others—the opposite polarity to the over-approximating, sound data-flow analysis of Section 3.2—and therefore warn, except the definite misconfiguration SC6003. Semantic checks verify consistency with supplied premises; they do not prove those premises true.

The IR itself is a recursive graph of named states with a designated start. Each state records its ASL kind, normal successors, **Catch** edges, the five-stage I/O pipeline, retry and timeout controls, JSONPath-vs-JSONata mode, nested **Map/Parallel** sub-machines, and the annotation slot. This is enough for every pass; surface syntax stays in the front ends.

3.2 Native Data-Flow Verification

The native layer is a sound bug finder, not a complete verifier. If STEPCheck reports SC1101, the referenced field is absent on every execution reaching that read. If it is silent, the field may still be absent for some inputs hidden behind **T**. This is the right direction for a raw-artifact checker facing arbitrary input and opaque task results.

Table 1. STEP CHECK’s eight passes. Native sound checks report errors without annotations; conservative checks report warnings except SC6003; semantic checks are relative to supplied facts.

Pass	Codes	What it catches	Claim
Structural and annotation hygiene	SC0001–0012	undefined targets, unreachable/dead-end states, non-exhaustive Choice , stale sidecar references	well-formedness; SC0007 policy warning
Data-flow provenance	SC1001–1003, SC1101, SC1110	missing-field references, binding faults, absent-operand Choice guards (warning)	sound bug finding (Thm. 1)
Concurrency	SC5001–5002	fan-out branches or iterations overwrite the same external resource	conservative warning
Temporal	SC6001–6003	unbounded callback, retry budget past timeout, heartbeat $\geq$ timeout	conservative; SC6003 error
Typed contract	SC1010	successor requires a field its immediate predecessor does not produce	declared-tier precise (Prop. 1)
Typestate	SC2001–2002	out-of-order task composition or illegal protocol edge	declared-tier precise (Prop. 1)
Retry-safety	SC3001	broad retry of a non-idempotent task	relative to idempotency premise
Saga compensation	SC4001–4002, SC4010–4011	missing/incomplete compensation for persistent effects	relative to declared model (Thm. 2)

The analysis tracks *document and variable shapes*: either $\top$ (opaque: any field may be present) or a finite record of fields that may occur. Joins union keys, with $\top$ absorbing; a path is definitely absent when record descent reaches a missing key before $\top$. The transfer functions mirror the ASL I/O pipeline: **InputPath/OutputPath** re-root the shape, **Parameters/ItemSelector** constructors bind static fields and checked $.\$$ reads, opaque task results become $\top$, **ResultPath** merges into raw input preserving siblings, **Assign** updates workflow-variable shapes, and **Catch** edges merge an opaque error object. For JSONata, STEP CHECK models direct $\$states.input$, $\$states.result$, and $\$var$ references plus object-valued **Output/Assign/Arguments** fragments; unsupported expressions project to $\top$ and do not produce SC1101.

A forward iteration joins shapes along normal and **Catch** edges. Cycles can re-embed a sub-document under a **ResultPath** and grow record depth without changing the key set, so STEP CHECK applies depth- k widening after every propagation step: any record nested deeper than k (the implementation uses

$k = 12$) is replaced by $\top$. Widening is conservative—it can hide a bug behind opacity, but it cannot manufacture a definite absence. With a finite key universe and bounded record depth, the lattice has finite height on every workflow topology. The implementation uses $|states| \times (|keys| + 2) \times (k + 1) + 2$ rounds as a safety bound and telemetry denominator; if a bug ever exceeded it, all known entries would be lifted to $\top$ before checking rather than reported from an under-approximation.

In practice the bound is not approached: across the 193 AWS, 66 CNCF, and two DSL workflows, all 790 data-flow fixpoints converge, with a maximum of 7 rounds and a worst observed bound utilization of 2/54 (3.7%).

Figure 1 marks the first ASL task, **Reserve Flight**, reading `$.flightTable` and `$.outboundFlightId` through `Parameters`; in the upstream template, those fields come from the AppSync resolver’s execution payload. With a generated or declared entry schema, omitting either field is a definite missing-field fault; with raw ASL input alone, the initial document is $\top$ and the native pass stays silent. Loops use the same abstraction: a state with a back-edge joins the pre-loop and loop-carried shapes, so SC1101 fires downstream only when *no* iteration can produce the read field.

Theorem 1 (Sound bug reports for missing-field references). *Under the modeled JSONPath/JSONata document-and-variable semantics, if STEPCheck reports SC1101 for a path p at state s , then every concrete execution reaching s lacks p in the document or variable selected by that read site, so the ASL runtime fails that reference. In the native tier this holds for arbitrary execution input; in the typed tier it holds relative to the closed-world reading of declared input schemas and, when result-shape modeling is enabled, declared task-output schemas and fixed AWS service-result envelopes. Expressions outside the modeled fragment lift to $\top$ and are therefore silent.*

Proof. Let γ map each abstract document/variable shape to the concrete JSON values it may denote, with $\top$ denoting all values and records denoting exactly their listed keys. Each transfer function over-approximates the corresponding ASL operation: constructors build closed records; opaque tasks, arrays, unsupported selectors, and unsupported JSONata expressions lift to $\top$; `ResultPath` and `Catch` place over-approximating shapes into the raw input; and `InputPath/OutputPath` either navigate inside the current abstraction or return $\top$ for unmodeled selection. Joins are least upper bounds, and depth- k widening is extensive because replacing a sub-record by $\top$ only enlarges γ . Therefore the forward iteration computes a post-fixpoint that over-approximates every concrete environment reaching each state, including loop iterations. If lookup of p at the relevant read site reaches a missing key before $\top$, then no concrete value in γ contains that path. The modeled ASL read therefore raises the runtime missing-reference error. Inputs and task results treated by declared schemas are closed-world premises; expressions outside the modeled fragment reach $\top$ and cannot trigger the premise of the theorem. $\square$

SC1110 applies the same definite-absence judgment to **Choice** guards but stays a warning outside Theorem 1: rule reachability depends on earlier guards

such as `IsPresent`, so it marks a provably dead or risky branch rather than an unconditional runtime fault.

3.3 Concurrency and Temporal Analyses

The two remaining native passes are conservative engineering checks. `Parallel` branches and concurrent `Map` iterations receive document copies, so only external resources can race. `STEPCHECK` records statically known resource keys (`TableName`, `QueueUrl`, `Bucket`, and other named targets; full list in the technical report). It emits `SC5001` only for provably overwriting operations—`PutItem`/`PutObject` and statically named `states:startExecution` collisions—leaving keyed, append-only, and compute-only operations silent.

The temporal pass checks resilience knobs the platform does not cross-check. `SC6001` warns when a callback or Activity task lacks a timeout or heartbeat and can wait to the platform maximum. `SC6002` warns when a retry budget can exceed the machine timeout. `SC6003` errors on the definite misconfiguration `HeartbeatSeconds ≥ TimeoutSeconds`.

3.4 Semantic Verification

The semantic layer consumes task facts ASL does not contain: a contract $T : A \rightarrow B$ (fields consumed and produced), a typestate transition $s_{\text{in}}^T \rightarrow s_{\text{out}}^T$ in protocol δ , idempotency (whether a retry repeats an external effect), and persistence with an optional compensator. These are premises (Section 3.5); `STEPCHECK` checks the workflow relative to them.

Proposition 1 (Declared-tier consistency). *For a fully declared workflow W , `SC1010`/`SC2001`/`SC2002` emit no finding iff every normal-flow task adjacency $T_1 \rightarrow T_2$ satisfies $\text{in}(T_2) \subseteq \text{out}(T_1)$ and $s_{\text{out}}^{T_1} = s_{\text{in}}^{T_2}$, and every task transition belongs to the declared protocol δ . On the series-parallel fragment, this edge condition is equivalent to a standard field/typestate typing judgment. Completeness is relative to the supplied annotations.*

Here $\text{out}(T_1)$ is T_1 ’s declared output set—including fields it forwards, not only those it computes—and $\text{in}(T_2)$ is T_2 ’s declared input set; under this closed-world convention the edge-local check does not false-positive on a field merged upstream and passed through. The checks are edge-local: a ship-before-pay reordering in the order workflow yields typestate mismatches plus field-contract misses. Retry-safety is likewise premise-relative: a broad `States.ALL`/`States.TaskFailed` retry of a task declared non-idempotent is `SC3001`, while retries narrowed to specific transient classes are left alone.

Compensation is about downstream error flow. Per-task checks (`SC4001`/`SC4010`) can see that a persistent task has no compensating path, but they miss a later state failing after the effect has committed. For a declared persistent task P with compensator C , `SC4011` scans the post-commit region reachable from P before C and asks whether every fallible state’s generic failure

routes through a broad `Catch` to C , and whether no normal path reaches `Fail` while bypassing C .

The declared Saga model assumes a single post-commit fault, feasible normal paths, a unique generic catcher for `States.TaskFailed`, and linear handlers whose reachability to C means execution of C . These assumptions exclude failing compensators and branching recovery handlers.

Theorem 2 (Relative soundness and completeness of the compensation check). *For a workflow with declared persistence and compensators, under the declared Saga model, SC4011 emits no finding for persistent task P with compensator C iff every modeled post-commit failure after P executes C . Thus SC4011 is sound and complete relative to the declared model.*

Outside these assumptions, SC4011 remains a reachability check but is no longer sound or complete. The model-independent SC4001/SC4010 checks still flag any persistent effect with no compensating path. Coverage therefore degrades to the per-task tier rather than going silent; the built-in and corpus sagas satisfy the assumptions.

3.5 Recovering Workflow Semantics

Declared facts enter through a sidecar for existing ASL or the typed DSL for greenfield workflows; once declared they support error-level findings, and where absent the declared-tier checks are silent.

Listing 1.1. A sidecar annotation (TOML), excerpt.

```
[tasks."Reserve Booking"]
idempotent = false
persistent = true
compensation = "Cancel Booking"
state_in = "SeatReserved"
state_out = "BookingReserved"
```

The booking saga’s complete sidecar is 69 lines of TOML—about eight per-task lines over four tasks, plus shared schema and protocol blocks—and the typed DSL emits the same facts by construction. Optional inference gives existing repositories a zero-effort warning mode: read-like names imply idempotent, write-like non-idempotent, resource-acquiring persistent, and notifications non-idempotent but not persistent; with no signal it abstains, and explicit annotations override it. Stale sidecar and schema references are caught syntactically (SC0011/SC0012). The residual risk, semantic drift between a declared premise and the deployed Lambda, is outside STEP CHECK’s scope; generating the sidecar from code or IaC keeps declared facts in sync.

4 Implementation

The framework of Section 3 is realised directly as `stepcheck`, a single dependency-light Rust command-line tool of **6,728** source lines (excluding a 900-line test

Table 2. Diagnostic codes by class, severity, and soundness tier. *Sound*: no false positives under the stated premise (Theorem 1). *Conservative*: a heuristic that abstains rather than over-reports. *Declared*: exact relative to supplied annotations. *Advisory*: inference-driven, non-blocking adoption aid (Section 5.2). Mixed severities mean the family contains both blocking errors and non-blocking warnings.

Code(s)	Class	Severity	Tier	Required premise
SC0001–SC0010	structural well-formedness	error/warn	sound	—
SC0011/SC0012	sidecar consistency	error	declared	supplied sidecar
SC1001–SC1003	native data binding	error/warn	sound	valid ASL grammar
SC1101/SC1110	data-flow field provenance	error/warn	sound	modeled fragment (else $\top$)
SC1010/SC2001/SC2002	contracts / typestate	error	declared	declared schemas/protocol
SC5001–SC5003	concurrency interference	warning	conservative	static resource/child key
SC6001–SC6003	temporal bounds	error/warn	sound	declared bounds
SC4002/SC4011	declared saga consistency	error	declared	declared compensator
SC3001/SC4001/SC4010	retry / compensation	error/warn	advisory/declared	inferred or declared effects

suite) that follows the pipeline of Figure 2, covering eight analysis passes, four front ends, the ASL emitter, and the self-validation tooling.

Table 2 maps each diagnostic family to its defect class, default severity, *soundness tier*, and required premise. This is the reader’s lookup table: it separates must-fix errors from warning-level adoption checks, and proven native facts from declared or advisory premises.

Two small extension interfaces realise the architecture: a **Frontend** lowers an input format to the IR, and a **Pass** analyses that IR and emits diagnostics. Behind them sit the typed DSL, raw ASL, CNCF Serverless Workflow [11], and a **CloudFormation / SAM / CDK** front end; all share the same IR and pass implementations (Section 5.9). The raw-ASL front end lowers generic JSON with a stack-growing deserializer, parsing all 193 corpus workflows and the nested-Map stress cases without error.

Running on the deployment artifact. A serverless team rarely commits a standalone ASL file: the state machine lives inside a CloudFormation or SAM template (a `DefinitionString` with `!Sub`/`!GetAtt` intrinsics) or is generated by CDK. The CloudFormation front end reads such a template—or a `cdk synth` output template—directly: it finds every `AWS::StepFunctions::StateMachine`, resolves the intrinsics (keeping the AWS pseudo-parameters exact so service-integration ARNs survive, mapping resource references to placeholder ARNs), follows local `DefinitionUri` references, and lowers each machine to the same IR. `stepcheck check template.yaml` and `stepcheck check cdk.out/Stack.template.json` therefore work with no manual extraction, which makes the build-time gate (Section 5.5) a drop-in check on the deployed artifact rather than on a separately prepared definition.

Running the verifier. STEPCHECK is a single self-contained binary, packaged for `crates.io` and `npm` (packaging details in the technical report). A developer runs `stepcheck check workflow.json` on an existing definition; `-infer` lights up the semantic checks from name and resource conventions, `-annot sidecar.toml`

supplies declared schemas/protocol/effects (Section 3.5), and `-result-shapes` enables closed output records for declared task results and common AWS service envelopes. Findings render for a terminal or, with `-json`, as a stable machine record (code, severity, state, message, remediation).

By default the exit status is 0 (clean), 1 (any *error*), or 2 (parse failure). The *recommended CI policy* fails a build on errors—the sound native and declared-tier findings—while leaving *inferred warnings* non-blocking. This lets a repository adopt STEPCHECK without triaging heuristic findings up front. `-deny-warnings` tightens the gate once a repository wants warnings to block. The artifact ships a runnable GitHub Actions gate (`.github/workflows/gate.yml`) that enforces this policy on every push/PR and self-tests, via an injected defect, that the gate actually blocks it.

Nothing else is deployed: the check reads the ingested ASL and never re-emits it, so raw-ASL verification is entirely independent of the emitter.

The remaining components validate the verifier itself. First, a data-flow certificate checker recomputes the widened invariant and checks finite proof obligations: the start invariant, every normal/catch transfer as a post-fixpoint condition, and every emitted SC1101 as a definite absence query in that invariant. This discharges loops by induction over the fixpoint rather than by bounded unrolling (Section 5.6). Second, a code-disjoint execution oracle re-interprets a workflow concretely along bounded loop-unrolled paths and cross-checks the same conclusion; this is an empirical differential check in the sense of test oracles and translation validation [8,19,22], not the proof itself. A mutation command injects one defect of a chosen class and re-emits deployable ASL for the study in Section 5.

Implementation limitations. Distributed `Map` item contexts are modelled coarsely. The ASL emitter—used as the DSL’s code generator and in evaluation harnesses—now carries uninterpreted ASL fields through the IR. Round-tripping 193 workflows preserves 100% of states and all measured I/O-processing fields (including `JSONPath`, `JSONata`, and `Map` item-selection fields), gives exact ASL-object equality for 99.5% of workflows, and leaves `emit` idempotent on all 193 (Section 5.8). This is an ASL round-trip guarantee; deployment wrappers such as CloudFormation metadata remain outside the emitter.

5 Evaluation

We evaluate STEPCHECK on real serverless workflows: RQ1, does it detect the defect classes of Figure 1? RQ2, what does verification cost? RQ3, does the shared-IR design extend beyond AWS? Controlled mutation studies measure per-check detection power; unmodified workflows, deployment templates, issue reports, and fixed-bug pairs provide real-world evidence.

5.1 Setup and corpus

We organize evidence by realism before size. The foreground set is public but deployment-shaped: six industrial-topology **aws-samples** workflows, including AWS Serverless Airline Booking **ProcessBooking**, a canonical serverless benchmark [2,15]; **55 serverless-patterns** workflows read from SAM/CloudFormation deployments; and **24** AWS Solutions Library artifacts (**26** machines), including CDK-synthesized enterprise orchestrators. These public artifacts are reproducible stand-ins for proprietary industrial definitions, not arbitrary toy repositories. Broader strata provide coverage and stress: **193** AWS sample workflows, **66** CNCF examples, and **95** independent ASL definitions.

The 193-workflow AWS stratum is mined exhaustively from every parseable ASL definition while excluding build-configuration JSON. It contains 1,355 state entries: 1,354 valid states (median 6, max 33) plus one malformed non-state **QueryLanguage** entry reported as SC0010. It has 166 **Choice**, 64 **Map**, and 33 **Parallel** states; 84 workflows with retries; and 51 with catch handlers. It also includes **5** JSONata-mode, **19** Distributed Map, and **18** nested-**states:startExecution** workflows across nine AWS services. All experiments use a Rust 1.96 release build on a commodity laptop.

5.2 Detection in the wild

Run over the corpus with optional inference enabled as a non-blocking adoption mode, **STEPCHECK** flags **99 of 193** workflows. The native findings are small but concrete. Nine workflows have a **Choice** with no **Default** (SC0007), a runtime-risk warning because an input matching no rule fails. One workflow nests a machine-level **QueryLanguage** directive inside **States** (SC0010). Another 33 callback tasks lack a timeout or heartbeat (SC6001); each can wait up to the platform limit while holding business state open. The sound SC1101, dead-guard SC1110, and concurrency checks are silent on the default corpus run; their power is exercised below.

The semantic warnings surface adoption targets: SC3001: 36, SC4001: 117, and SC4010: 28. These are an *advisory* adoption aid, not a verification claim—the paper’s evidentiary weight is on the label-free sound native tier (SC1101, Theorem 1) and the declared tier. To evaluate them without leakage, we *freeze* the inference rules (pinned by their source blob hash), and split the human-labelled gold (23 workflows, double-labelled by two annotators) into a **design** partition of 10 workflows that informed rule authoring and a **hold-out** of 13 labelled afterward. On the hold-out, per-check inter-annotator agreement is high ($\kappa = 0.97$ idempotency, 0.98 persistence) and warning precision is **27/29 (93%)** for SC4001, **3/3** for SC3001, and **4/4** for SC4010; the underlying property inference is high-recall but moderate-precision (idempotency recall 0.94, precision 0.56; false-omission rate 0.06), exactly the profile of an adoption aid that prefers to surface a risk over hiding one. The downstream declared check SC4011 is silent on the wild corpus because no workflow declares a compensator. The declared Saga fixtures contain 7 persistent steps; **STEPCHECK** accepts the complete compensation

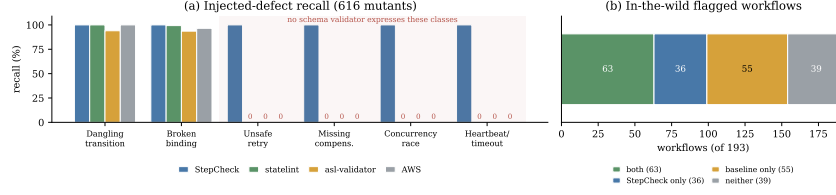


Fig. 3. Detection versus the validator panel: (a) recall over 616 mutants—STEPCHECK detects all six classes, existing validators none of the four semantic ones; (b) mostly disjoint in-the-wild coverage over the 193 workflows.

Table 3. Injected-defect detection over 616 mutants. A mutant counts as detected only when a tool emits a new problem naming the injected fault; incidental schema nits are not credited.

Defect class	Check	Appl.	STEPCHECK	Baseline validators		
				statelint	asl-val.	AWS
Dangling transition	SC0002	166	166 (100%)	166 (100%)	156 (94%)	166 (100%)
Broken data binding	SC1003	141	141 (100%)	140 (99%)	132 (94%)	136 (96%)
Unsafe retry	SC3001	103	103 (100%)	0	0	0
Missing compensation	SC4001	19	19 (100%)	0	0	0
Concurrency interference	SC5001	21	21 (100%)	0	0	0
Temporal heartbeat/timeout	SC6003	166	166 (100%)	0	0	0
Total		616	616 (100%)	306 (50%)	288 (47%)	302 (49%)

chains and, on the injected chain-break case, reports exactly `CreateOrder` and `ReserveInventory`, the upstream effects that would leak.

5.3 Detection power versus existing validators

For each applicable workflow, `stepcheck mutate` injects one defect and re-verification counts a detection only when the expected diagnostic increases over the unmutated baseline. Across **616** mutants in six corpus-scale classes, STEPCHECK detects every injected defect (Figure 3a and Table 3). This is per-check reliability on targeted operators, not a claim of complete bug coverage.

We compare against the validators developers already use—`statelint`, `asl-validator`, and AWS’s authoritative `ValidateStateMachineDefinition`. This is a complementarity test, not a superiority claim: those validators catch schema-expressible dangling transitions and broken JSONPath syntax, but do not attempt unsafe retries, missing compensations, races, or heartbeat/timeout violations. In the wild the two flag largely disjoint workflows (Figure 3b): STEPCHECK alone flags 36, the panel alone 55, mostly schema nits.

5.4 Comparison with three formal workflow verifiers

Schema validators are the tools a developer reaches for, but they are not a research-grade adversary. We therefore also compare against *three* formal workflow-

Table 4. STEPcHECK versus three formal verifiers (Woflan; BPMN Analyzer 2.0; BProVe) over 193 workflows, per-class recall with exact 95% CIs. The tools are complementary: all overlap on SC0002; STEPcHECK is unique on the data-flow / atomicity / concurrency / temporal classes with no workflow-net expression.

Defect class	Check	n	STEPcHECK (CI)	Woflan	BPMN-Anlz.	BProVe
Dangling transition	SC0002	166	1.00 [.98,1]	0.95	0.87	0.69
Broken data binding	SC1003	141	1.00 [.97,1]	0.00	0.00	0.00
Unsafe retry	SC3001	103	1.00 [.97,1]	0.00	0.00	0.00
Missing compensation	SC4001	19	1.00 [.82,1]	0.37	0.37	0.11
Concurrency interference	SC5001	21	1.00 [.84,1]	0.00	0.00	0.05
Temporal heartbeat/timeout	SC6003	166	1.00 [.98,1]	0.00	0.00	0.00
Formal-verifier over-report on <i>valid</i> wf.				16/193 (8.3%)	16/193 (8.3%)	56/193 (29%)

soundness verifiers, run locally: **Woflan** [28], the classical workflow-net soundness verifier of van der Aalst and Verbeek (via `pm4py`); **BPMN Analyzer 2.0** [20], a recent BPMN-specific model checker of Kräuter et al. (a Rust reachability-graph checker with counterexamples); and **BProVe** [12], via its BPMNOS BPMN parser plus Maude model checker. The baselines decide the standard workflow-soundness subproperties—*safeness*, *option to complete*, *proper completion*, and, where exposed, *no dead activities*. We encode each ASL workflow into a BPMN workflow net (rules A–E: Choice $\rightarrow$ exclusive gateway, Parallel $\rightarrow$ parallel split/join, Map $\rightarrow$ activity, Catch $\rightarrow$ error gateway, terminals merged to one end event), and run the same confound-free protocol: `emit` control versus `mutate` mutant, crediting a verifier only when the mutant encoding carries a fresh soundness violation.

The result is *complementarity, not superiority* (Table 4). On the one control-flow class all can express, dangling transitions (SC0002), the verifiers detect fresh violations at different recall levels: Woflan 95%, BPMN Analyzer 87%, and BProVe 69%. On the classes that are not workflow-net soundness properties—data binding, unsafe retry, and heartbeat/timeout—all are silent by construction while STEPcHECK detects all; SC1101 data-flow has no BPMN expression at all. Compensation and concurrency appear only as structural side effects for some baselines (Woflan/BPMN Analyzer 7/19 compensation; BProVe 2/19 compensation and 1/21 concurrency). Crucially, the baselines over-report on valid AWS serverless workflows: Woflan and BPMN Analyzer each flag **16 of 193** (8.3%), and BProVe flags **56 of 193** (29%), because independent, multi-terminal branch structure is not always a well-handled workflow net—a semantic mismatch between classical soundness and ASL, not a STEPcHECK finding. Confidence intervals are exact (Clopper–Pearson); no recall is a bare “100%”, and even on the small semantic classes Fisher exact separates STEPcHECK from each verifier at $p < 4 \times 10^{-5}$.

BProVe’s soundness checking addresses BPMN soundness and incidental structural effects, while STEPcHECK’s ASL-native data-flow covers SC1101; thus SC1101 is StepCheck versus out-of-scope, not a BProVe miss. Likewise, SC6003 is a native ASL temporal bound, not a BPMN dead-activity property.

Table 5. SAB `ProcessBooking`: per-class detection on one applicable mutant per class.

Defect class	Check	STEPCHECK	Woflan	BPMN-Anlz.	BProVe
Dangling transition	SC0002	Yes	Yes	Yes	Yes
Broken data binding	SC1003	Yes	No	No	No
Unsafe retry	SC3001	Yes	No	No	No
Missing compensation	SC4001	Yes	No	No	No
Concurrency interference	SC5001	n/a	n/a	n/a	n/a
Temporal bound	SC6003	Yes	No	No	No

Because Figure 1 uses AWS Serverless Airline Booking (SAB), we also run the same three-verifier protocol on `ProcessBooking` alone (Table 5). The clean SAB workflow is accepted by Woflan, BPMN Analyzer, and BProVe. On applicable SAB mutants, all tools catch the purely structural dangling transition, but only STEPCheck catches the ASL-native binding, retry, compensation, and temporal defects; SAB has no applicable concurrency-interference site. This is not a statistical claim, but a benchmark-specific witness that the complementarity in Table 4 holds on the paper’s canonical industrial proxy.

The quantitative axis is orthogonal. Both baselines above check *control-flow soundness*. A different, quantitative line of work verifies workflow *performance*: Su and Liu [24] model AWS Step Functions as semi-Markov chains and compute first-passage-time (FPT) moments—up to the fourth moment for a 10,000-state model in ≈ 70 s—to profile execution-time distributions. This is *complementary and orthogonal* to STEPCheck: our temporal checks are static, single-mode τ -bounds (SC6001/SC6003: is a callback bounded, is a heartbeat consistent with its timeout), whereas Su and Liu’s are probabilistic multi-order FPT moments over a stochastic model. STEPCheck targets temporal *correctness* at build time, not SLA *prediction*; the two answer different questions and could compose (a static gate feeding a quantitative model).

5.5 Industrial case study and build-time gate

We assemble an industrial-topology set of six workflows: four `aws-samples` production patterns (a travel-reservation saga with retry on every forward task and a three-step compensation chain, a CDK saga, an inventory reserve-stock saga, and a checkout workflow), a Map-based ETL aggregator, and the AWS Serverless Airline Booking `ProcessBooking` saga (12 states: reserve flight/booking $\rightarrow$ collect-payment $\rightarrow$ confirm $\rightarrow$ notify, with release-seat / cancel-booking / refund-payment compensation and a booking DLQ), read *directly* from the SAM `template.yaml` in the project’s `master` branch by the CloudFormation front end—`stepcheck check src/backend/booking/template.yaml` needs no manual extraction. On these sagas STEPCheck surfaces advisory retry-safety and compensation-coverage findings (SC3001/SC4001) on durable steps, while Woflan reports all six sound. The

Table 6. Build-time gate on the industrial set: with M defects injected, StepCheck’s build-fail catch-rate vs. the platform validator’s silent pass-through (accepts $\Rightarrow$ would deploy), and gate latency. The gate catches what the platform deploys, at CI cost.

Injected defects	STEPCHECK gate catch	Platform silent-pass	Gate p50/p95/p99 (ms)
0 (clean)	—	0.83	12.5/15.9/28.0
1 (semantic)	1.00	0.80	11.8/17.1/29.5
5 (mixed)	1.00	0.00	13.0/21.4/26.6

same fair-class protocol reproduces the complementarity of Table 4 (STEPCHECK 100% per class; Woflan only SC0002).

As a build-time gate, `stepcheck check --deny-warnings` runs at CI cost. We inject 0/1/5 defects at build time and report, per level, the gate’s build-fail catch-rate, the platform validator’s silent pass-through (`asl-validator`, the schema check AWS’s `ValidateStateMachineDefinition` performs—does the mutant still “validate” and deploy?), and the gate latency (Table 6). The practitioner-facing result: at a single injected *semantic* defect the gate fails the build on **every** workflow while the platform accepts 80% of them (the defect would deploy); at five defects the gate still catches all, and the platform only rejects the ones with a co-injected *syntactic* fault while silently passing the semantic ones. Gate latency is stable in the defect count—p95 16–21 ms end-to-end including process start—so the gate is a genuine drop-in (configuration in the artifact).

5.6 Data-flow provenance: soundness, power, and reach

The central SC1101 claim is no false positives. In the default native corpus run it fires zero times. To exercise its power, the typed-tier mutator is seeded with declared input fields and injects a missing-field read into each of the **126** workflows with a suitable site. With service/output shape modeling enabled (`-result-shapes`), STEP CHECK detects **88 of 126 (70%)**. An independent certificate pass then discharges the emitted reports from a finite widened-fixpoint invariant: across **207** analyzed machines it checks **1,146** normal/catch post-condition obligations, certifies all **88** emitted SC1101 reports, and finds zero failed obligations or diagnostic mismatches. A separate concrete execution oracle remains as a code-disjoint empirical check: on the same configured population it reports zero present-field counterexamples on bounded loop-unrolled reaching paths.

The check is not vacuous: it catches the **amount/total** miss validators accept, and flow-sensitive misses through `ResultPath` merges that adjacent-task field-subset checks miss. The implementation resolves dotted and quoted-bracket member paths; `-result-shapes` treats declared task-output schemas as closed-world result records and adds closed top-level envelopes for common AWS integrations (Lambda invoke, DynamoDB, SNS, SQS, EventBridge, and Step Functions child

execution). The remaining misses are behind unsupported or templated service results, arrays, or complex selectors that still lift to $\top$.

The modeled fragment covers most real reads: of the corpus’s genuine document-field references, **92%** use dotted access and resolve precisely, and **8%** use bracket/wildcard/filter syntax abstracted to $\top$ ($\$$ context and `States.*` operands are not document references; `stepcheck path-coverage`, full breakdown in the technical report).

5.7 Real defects in the wild

We mined fix-commit pairs whose diff edits an ASL definition and ran STEPCHECK on both revisions. In **7 of 39** pairs a finding is present only on the buggy revision; adjudication confirms **6** genuine defects across five repositories, one an official AWS sample (Table 7); four are semantic/logic classes no schema validator expresses. This is existence evidence, not a prevalence estimate.

Beyond fix-commit diffs, we also searched issue trackers for user-reported defects: closed GitHub issues whose quoted runtime error is exactly the fault STEPCHECK’s sound tier prevents. Three are textbook SC1101, one in an *official AWS workshop: serverless-coffee-workshop* #56 (“The JSONPath `$.detail.orderId` ... could not be found in the input `{"orderNumber": "10"}`”, filed by a user who reran the lab three times), *campus-compute* #32 (an audit step reads a job field wiped by a preceding SNS `Publish` whose default `ResultPath` overwrites the document), and *turbofan* #1 (a fan-out reads `$.input`, absent from the trigger payload). These machines are built in CDK, but CDK synthesizes ASL: for #32 we checked out the buggy commit, ran `cdk synth`, and extracted the *real* 14-state definition—on which STEPCHECK flags exactly SC1101 at `AuditLogFailed` reading `$.execution_id`, the reported fault. The others are confirmed on a minimal reproduction of the reported structure with the issue URL and verbatim error recorded. These are user-filed defects, not author-selected diffs, and SC1101 is the most issue-tracker-visible class because it fails with an explicit JSONPath error. A second pass over standalone ASL files from independent repositories surfaces a real SC3001 in *solo-vault-backend*’s pipeline (an SQS send, non-idempotent, retried on `TaskFailed`); a *data-meshy* subscription saga has proper compensation and produces no finding.

5.8 Cost, overhead, and deployment

Verification is cheap: the eight passes average $\approx 39 \mu\text{s}$ per workflow (median $\approx 25 \mu\text{s}$, max $\approx 1.1 \text{ ms}$), and the check adds no runtime component because it runs before deployment. This corpus is not only acyclic examples: it contains **84** workflows with `Retry`, **52** with `Map`, **29** with `Parallel`, **55** with control-flow cycles, and nesting depth up to three. To check cost on production-shaped artifacts, we also timed the real industrial set (6 workflows, 65 recursive states, retries/catches plus `Parallel` and `Map` topologies): mean $\approx 91 \mu\text{s}$ per workflow, median $\approx 100 \mu\text{s}$, max $\approx 0.17 \text{ ms}$, total $\approx 0.55 \text{ ms}$. Across the AWS Solutions corpus (24 workflows/artifacts, 385 recursive states), mean cost is $\approx 0.28 \text{ ms}$ per workflow

Table 7. Confirmed in-the-wild defects re-discovered on deployed artifacts.

Code	Repository	Defect fixed
SC1101	aws-batch-runtime-monitoring	document field used instead of context input
SC1101	serverless-ai-fitness	tasks read fields never produced
SC1110	serverless-ai-fitness	subscriber branch unreachable
SC6001	leave-management	callback can hang indefinitely
SC0002	airway-shipment-orchestrator	transition targets missing state
SC0003/4/5	video-translation	string End leaves dead states

(median ≈ 0.12 ms, max ≈ 1.58 ms, total ≈ 6.70 ms). The CDK-synthesized AWS Solutions subset used for gate timing includes 173 recursive states, five **Map**-bearing templates, four cyclic workflows, and one nested-**Map** template (depth two); running the checker as a CI process on these templates costs p50/p95 $\approx 15.7/20.9$ ms, while the smaller industrial set costs $\approx 9.4/11.3$ ms. Synthetic chains and stress shapes remain scalability sanity checks: 30,000 acyclic states verify in ≈ 0.13 s, a 512-branch **Parallel** fan-out (2,051 states) in ≈ 49 ms, and a 160-level nested **Map** in ≈ 11 ms.

The DSL stays concise (20 declarations expand to a 119-line annotated ASL definition). Our live deployment check spans three real AWS execution models, each deploying a StepCheck-verified definition unmodified (only **FunctionName** rebound to a stand-in Lambda) to **SUCCEEDED**: the order workflow as a synchronous *Express* machine; the same definition as an asynchronous *Standard* machine, whose durable history retains all **24** events (Express retains none); and a DSL-verified order variant with a bounded `waitForTaskToken` approval gate—which Express cannot run—deployed as *Standard*, where the run *pauses* at the callback and an external **SendTaskSuccess** resumes it to completion (**30** events). The deployed callback draws no **SC6001** because its **TimeoutSeconds** bounds the wait; removing that bound makes the temporal pass flag it pre-deployment. The verified artifact also runs at scale: across **100** executions in each non-callback mode it completes **100/100** times—Express at p50/p95 $\approx 72/130$ ms and *Standard* (durable, five state transitions per run) at p50/p95 $\approx 345/456$ ms of execution time. This is a completion check across execution models, not a cross-mode performance benchmark. Scripts, per-run timings, and captured runtime logs ship with the artifact, together with a self-testing GitHub Actions gate (`.github/workflows/gate.yml`) that runs the blocking check on every push and asserts, via a negative test, that an injected defect fails the build. The emitter that makes the DSL-first story deployable is measured on the full 193-workflow ASL corpus: round-tripping through the IR (ASL $\rightarrow$ IR $\rightarrow$ ASL) preserves 100% of states and all measured I/O-processing fields (including **InputPath**, **OutputPath**, **ResultPath**, **ResultSelector**, **Parameters**, **Result**, **JSONata Arguments/Assign/Output**, and **Map** item-selection fields), achieves

exact ASL-object equality for 99.5% of workflows, and keeps `emit` idempotent on all 193. The single non-exact case is a malformed corpus entry with a non-state marker inside `States`, which the parser normalizes away rather than re-emitting.

5.9 Cross-format generalization: CNCF Serverless Workflow

The CNCF front end parses 66 Serverless Workflow examples and lowers them to the same IR. Structural, typestate, retry, compensation, concurrency, and temporal checks apply unchanged. Pure jq field references (`.a.b`, `${ .a.b }`) lower to the same JSONPath read by the sound SC1101/SC1110 checks, so field-provenance analysis fires at CNCF declared-record boundaries—a jq read of a field absent from a task’s declared input schema is flagged. A comparison or presence guard over a document field (`.a.b == v`, `.a.b < v`, `.a.b != null`, and the leading operand of a short-circuiting `and/or`) likewise lowers to that field’s read, while the remaining composite jq (pipes, arithmetic, object construction, and `$context/$workflow` runtime references) soundly projects to $\top$. Of the 66 workflows, 12 contain such modelable references (18 reference sites); the rest are schema-free and stay conservative. Mutation detects every injected control-flow defect with a site (retry 4/4, structural 9/9, concurrency 1/1, temporal 50/50), and where CNCF declares JSON Schemas SC1010 uses them exactly: a reordered schema-annotated order workflow is rejected with three contract errors.

5.10 External validity: independent repositories

To test whether the analyses overfit AWS’s own samples, we ran STEPCHECK on 95 further ASL definitions from 16 public repositories outside the AWS sample collections (50 domain workflows across 13 repositories, the rest validator fixtures; script and manifest in the artifact). The sound SC1101 fires on a credit-card transaction-reversal workflow: its `Pass` state reconstructs the document through `Parameters` into a closed record that drops the `ledgerEntries` field a downstream `Map` iterates. This is a flow-sensitive miss no adjacent-field check sees. The concurrency check flags an IoT firmware-upgrade workflow whose three `Parallel` branches write one DynamoDB table. Declaring the two durable effects of an independent payment-settlement pipeline through a sidecar promotes STEPHECK’s finding to a declared-tier *error*: a settlement failure leaves an authorized hold that is never voided. These hits, on workflows we neither authored nor curated, show the checks generalize beyond the sample corpus.

Consolidation on deployment artifacts. Because the CloudFormation front end reads the artifact a team actually ships, we further consolidate external validity on `aws-samples/serverless-patterns`, a recognised collection of real AWS serverless reference patterns. Running STEPHECK directly on their SAM/CloudFormation templates and `statemachine/` definitions—resolving intrinsics and following `DefinitionUri` with no manual extraction—covers 55 distinct pattern workflows. It flags 13, including two sound-tier findings (a `Choice` with no `Default`, SC0007; an unbounded callback, SC6001) and advisory

retry / compensation concerns (SC3001, SC4001/SC4010) on production-shaped patterns. We also run STEP CHECK on the **AWS Solutions Library**, an enterprise-deployed, partly CDK-generated class: *Media2Cloud* (16 committed state machines, 123 states, **Parallel** fan-out to child machines and nested **Maps**), *Customizations for AWS Control Tower* and *Network Orchestration for AWS Transit Gateway* (read from committed CloudFormation templates), plus four CDK-synthesised solutions: *Instance Scheduler*, *Automated Security Response*, *Distributed Load Testing*, and *Account Assessment*. Across the **26** machines STEP CHECK flags **14**, including native findings on enterprise orchestrators (a **Choice** with no **Default**, SC0007) and an unbounded callback in *Automated Security Response* (SC6001), plus advisory retry / compensation concerns on persistent create/delete/publish/send steps (SC3001/SC4001). *Instance Scheduler*, *Distributed Load Testing*, and *Account Assessment* are clean true negatives that exercise the CDK-synth path end-to-end. Repeating the build-time gate protocol on the six CDK synth templates, with mutants embedded back into the templates, catches all applicable mutations (6/6 at one defect, 5/5 at five); on the two CDK definitions **asl-validator** accepts cleanly, it silently passes both one-defect mutants, while gate p95 stays below 20 ms. The result-shape model treats **.waitForTaskToken** callback payloads as $\top$, because the callback result is supplied by the external actor rather than by the base service envelope. This moves the corpus from curated ASL to the deployment forms verification must actually meet.

Threats to validity. STEP CHECK verifies workflow definitions, not opaque Lambda or managed-service internals. Opaque results and JSONata expressions outside the modeled direct-reference/object fragment lift to $\top$, preserving soundness but losing precision. The mutation operators are our own, so each per-class recall is per-operator reliability against a targeted fault population, not detection power against an independent one; we therefore report it with exact Clopper–Pearson intervals (e.g. SC5001 21/21, CI [0.84, 1]) and never as a bare “100%”. Even on the small semantic classes the separation from the formal competitor is significant despite the sample size: Fisher exact of STEP CHECK versus Woflan gives $p < 4 \times 10^{-5}$ for SC4001 ($n=19$) and $p < 4 \times 10^{-12}$ for SC5001 ($n=21$). The mutants of the industrial case study (Section 5.5) are injected on workflows we did not author, and the in-the-wild and fixed-bug results (Sections 5.2, 5.7, 5.10) supply the complementary uncontrolled evidence. Declared-tier semantic results are relative to supplied annotations, whose truth is outside STEP CHECK’s scope. The data-flow certificate checker verifies finite post-fixpoint obligations for the widened invariant, so loop soundness is argued by induction over the invariant rather than by bounded path exploration. The execution oracle is a complementary empirical cross-check: it explores bounded loop-unrolled paths and would report any present-field counterexample it finds (Section 3.2). Proprietary industrial definitions remain excluded, and the public corpora still skew small by median workflow size; the public deployment-shaped sets mitigate rather than eliminate this threat, while generated sweeps remain secondary scale/topology stress rather than evidence of industrial semantic diversity (extended limitations

in the technical report). The concurrency pass composes across `startExecution`—two branches invoking the same child workflow are flagged (SC5003, two real corpus instances)—and flattens the child’s own write set when the definition is in scope. The CloudFormation/SAM front end links sibling machines through logical ids, `Ref/GetAtt`, `DefinitionSubstitutions`, and `StateMachineName`-derived ARNs; only externally deployed children whose definitions are absent from the artifact remain a scope boundary. The deployment check covers Express, Standard-mode async, and a live `.waitForTaskToken` callback resumed out-of-band, but on a single verified workflow family rather than a broad execution benchmark.

6 Related Work

Runtime orchestration and durable execution. Triggerflow rethinks how serverless workflows run [6], while durable-execution runtimes—Azure Durable Functions [10], Temporal [26], and Beldi’s transactional guarantees for stateful serverless workflows [29]—enforce recovery and transactions *during* execution. STEPCHECK is complementary: it statically checks the deployed Step Functions artifact *before* deployment with no new runtime, pursuing the same failure-atomicity goal from the opposite end.

Formal verification at other layers. AWS uses TLA^+ to model-check service designs, workflow soundness has been studied with Petri nets and service-composition model checking [21,1], and Zelkova brings SMT reasoning to IAM access policies [7]. These verify an abstract model, the platform, or a different artifact; STEPCHECK checks the executable ASL definition with linear passes suitable for CI. Closest to our retry-safety check, Flux verifies idempotence of stateful serverless functions [14] and could *supply* the per-task idempotency premises STEPCHECK’s SC3001 consumes.

Behavioural types and Sagas. Typestate and session types provide the basis for protocol conformance [23,17], and the Saga pattern is the standard compensation model [16]; STEPCHECK adapts both to serverless task graphs as edge-local declared checks and a declared-model reachability check.

Data-flow and cloud validators. Existing Step Functions validators check schema, path syntax, or execute tests, but not field provenance [5,9,3,4]. Data-flow anomalies have been studied for BPMN and BPEL [25,27], but there data is abstracted to named elements in a hand-built model; STEPCHECK tracks nested JSON document shapes through ASL’s I/O pipeline as an abstract interpretation [13]. Serverless formal foundations motivate the setting but do not provide static workflow verification [18].

7 Conclusion

STEPCHECK shows that useful static verification can run directly on the AWS Step Functions artifact developers already deploy. Its native data-flow analysis reports only definite missing-field failures. Its layered annotations make retry, typestate, and Saga checks precise relative to supplied facts. On real workflows it

rediscovers independently fixed bugs, detects targeted semantic mutants missed by existing validators, and runs fast enough for CI.

The sound core and the higher-stakes semantic checks sit in different tiers by design. Missing-field faults are caught with certainty on the raw artifact; the rediscovered and in-the-wild SC1101 bugs show how they escape testing behind **Choice** branches. Retry, race, and compensation faults instead begin as provenance-graded warnings and harden into declared-tier errors as effects are annotated or inferred. The layering is a migration path, not a fixed line. Serverless orchestration need not wait until execution to expose data, ordering, retry, timing, and compensation mistakes.

Artifact availability. An anonymized artifact—STEPCHECK, the corpus, scripts, figures, and reproduction instructions—is at <https://anonymous.4open.science/r/StepCheck>. The paper is self-contained; the artifact adds a companion technical report with the full formal development, algorithms, feature matrices, and extended limitations.

References

1. van der Aalst, W.M.P.: The application of Petri nets to workflow management. *Journal of Circuits, Systems and Computers* **8**(1), 21–66 (1998)
2. Amazon Web Services: AWS Serverless Airline Booking. <https://github.com/aws-samples/aws-serverless-airline-booking> (2019), public aws-samples application; accessed 2026-07-08
3. Amazon Web Services: ValidateStateMachineDefinition — AWS Step Functions API reference. https://docs.aws.amazon.com/step-functions/latest/apireference/API_ValidateStateMachineDefinition.html (2024)
4. Amazon Web Services: AWS step functions developer guide (amazon states language). <https://docs.aws.amazon.com/step-functions/> (2026)
5. Amazon Web Services Labs: statelint: A validator for Amazon states language documents. <https://github.com/aws-labs/statelint> (2024)
6. Arjona, A., García López, P., Sampé, J., Slominski, A., Villard, L.: Triggerflow: Trigger-based orchestration of serverless workflows. *Future Generation Computer Systems* (2021)
7. Backes, J., Bolignano, P., Cook, B., Dodge, C., Gacek, A., Luckow, K., Rungta, N., Tkachuk, O., Varming, C.: Semantic-based automated reasoning for AWS access policies using SMT. In: *Formal Methods in Computer-Aided Design (FMCAD)*. pp. 1–9 (2018)
8. Barr, E.T., Harman, M., McMinn, P., Shahbaz, M., Yoo, S.: The oracle problem in software testing: A survey. *IEEE Transactions on Software Engineering* **41**(5), 507–525 (2015). <https://doi.org/10.1109/TSE.2014.2372785>
9. Bougère, C.: asl-validator: A JSON-schema validator for Amazon states language documents. <https://github.com/ChristopheBougere/asl-validator> (2024)
10. Burckhardt, S., Gillum, C., Justo, D., Kallas, K., McMahon, C., Meiklejohn, C.S.: Durable functions: Semantics for stateful serverless. *Proc. ACM Program. Lang.* **5**(OOPSLA) (2021)
11. Cloud Native Computing Foundation: Serverless workflow specification. <https://serverlessworkflow.io/> (2026)

12. Corradini, F., Fornari, F., Polini, A., Re, B., Tiezzi, F., Vandin, A.: A formal approach for the analysis of BPMN collaboration models. *Journal of Systems and Software* **180**, 111007 (2021)
13. Cousot, P., Cousot, R.: Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. In: *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*. pp. 238–252. ACM (1977)
14. Ding, H., Wang, Z., Shen, Z., Chen, R., Chen, H.: Automated verification of idempotence for stateful serverless applications. In: *USENIX Symp. on Operating Systems Design and Implementation (OSDI)* (2023)
15. Eismann, S., Costa, D.E., Liao, L., Bezemer, C.P., Shang, W., van Hoorn, A., Kounev, S.: A case study on the stability of performance tests for serverless applications. *Journal of Systems and Software* **189**, 111294 (2022). <https://doi.org/10.1016/j.jss.2022.111294>
16. Garcia-Molina, H., Salem, K.: Sagas. In: *ACM SIGMOD Int. Conf. on Management of Data*. pp. 249–259 (1987)
17. Honda, K., Yoshida, N., Carbone, M.: Multiparty asynchronous session types. In: *ACM SIGPLAN-SIGACT Symp. on Principles of Programming Languages (POPL)*. pp. 273–284 (2008)
18. Jangda, A., Pinckney, D., Brun, Y., Guha, A.: Formal foundations of serverless computing. *Proc. ACM Program. Lang.* **3**(OOPSLA), 1–26 (2019)
19. Klinger, C., Christakis, M., Wüstholtz, V.: Differentially testing soundness and precision of program analyzers. In: *International Symposium on Software Testing and Analysis (ISSTA)*. pp. 239–250 (2019)
20. Kräuter, T., Stünkel, P., Rutle, A., Lamo, Y., König, H.: BPMN Analyzer 2.0: Instantaneous, comprehensible, and fixable control flow analysis for realistic BPMN models. In: *International Conference on Business Process Management (BPM)* (2024)
21. Newcombe, C., Rath, T., Zhang, F., Munteanu, B., Brooker, M., Deardeuff, M.: How Amazon web services uses formal methods. *Communications of the ACM* **58**(4), 66–73 (2015)
22. Pnueli, A., Siegel, M., Singerman, E.: Translation validation. In: *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*. LNCS, vol. 1384, pp. 151–166 (1998)
23. Strom, R.E., Yemini, S.: Typestate: A programming language concept for enhancing software reliability. *IEEE Transactions on Software Engineering* **SE-12**(1), 157–171 (1986)
24. Su, G., Liu, L.: Workflow trace profiling and execution time analysis in quantitative verification. *Future Internet* **16**(9), 319 (2024). <https://doi.org/10.3390/fi16090319>
25. Sun, S.X., Zhao, J.L., Nunamaker, J.F., Sheng, O.R.L.: Formulating the data-flow perspective for business process management. *Information Systems Research* **17**(4), 374–391 (2006)
26. Temporal Technologies: Temporal: Durable execution and workflow orchestration. <https://temporal.io/> (2026)
27. Trcka, N., van der Aalst, W.M.P., Sidorova, N.: Data-flow anti-patterns: Discovering data-flow errors in workflows. In: *Int. Conf. on Advanced Information Systems Engineering (CAiSE)*. LNCS, vol. 5565, pp. 425–439 (2009)
28. Verbeek, H.M.W., Basten, T., van der Aalst, W.M.P.: Diagnosing workflow processes using Woflan. *The Computer Journal* **44**(4), 246–279 (2001)

29. Zhang, H., Cardoza, A., Chen, P.B., Angel, S., Liu, V.: Fault-tolerant and transactional stateful serverless workflows. In: USENIX Symp. on Operating Systems Design and Implementation (OSDI) (2020)